Physics Informed Neural Network for UAV State Estimation

Nafis Mohammad Nayeem, Dr. Imraan A Faruque, Dr. Martin Hagan

Introduction

- **Why PINN was chosen?**

Physics-Informed Neural Networks combine **data-driven learning** with **known physical models (PDEs/ODEs)**
System dynamics are embedded **directly into the training loss** via physics residuals

Results in estimates that:

- Conform to physical laws by design
- Require less data than black-box models
- Provide a flexible, mesh-free framework for forward and inverse problems

PINNs act as physics-regularized state estimators, enabling robust and data-efficient UAV state awareness under noisy, GPS-denied, and partially observed conditions.

Big picture: Research on State estimation comparison

Extended Kalman Filter (EKF)
Invariant EKF
Physics-Informed Neural Network (PINN)
Journal is to be submitted to: Proc IMechE Part G (Journal of Aerospace Engineering)

Motivation and Objective

- **Why this matters (Aerospace relevance)**

Noisy / GPS-denied environments

Indoor flight, enemy jamming, Mars-like missions

PINNs rely on fundamental motion equations + minimal IMU data

Physics regularization limits drift when measurements degrade

Swarm coordination

Each UAV runs a local PINN estimator

Shared physics constraints (formation, collision avoidance) enhance cooperative estimation

- **How it is used here (UAV state estimation)**

Simulation-based data generation

12 UAV states simulated using kinematics and dynamics equations

Gaussian noise added to mimic imperfect sensors

PINN estimator

Neural network approximates the UAV state trajectory

Physics loss enforces flight-dynamics equations

Data loss enforces consistency with noisy sensor measurements

Data Generation

UAV dynamics
- Kinematics + Dynamics Equations
- Control → Forces & Moments

True States
- Ground truth (only available in simulation)

Gaussian Noise Injection
- Models sensor imperfections

Noisy States
- Stored as training data

PINN Learning
- While enforcing physics as soft-constraint

1. $\dot{p}_N, \dot{p}_E, \dot{h}$ — position in inertial (NED) frame
2. $\dot{u},\ \dot{v},\ \dot{w}$ — body velocities
3. $\dot{\phi},\ \dot{\theta},\ \dot{\psi}$ — Euler angles
4. $\dot{p},\ \dot{q},\ \dot{r}$ — body angular rates

where the forces and moments now explicitly depend on $\delta_e,\ \delta_a,\ \delta_r,\ \delta_t$. Symbolically:

$$\dot{p}_N = f_N(u,v,w,\phi,\theta,\psi), \quad \dot{p}_E = f_E(u,v,w,\phi,\theta,\psi), \quad \dot{h} = f_h(u,v,w,\phi,\theta,\psi)$$

$$\dot{u} = rv - qw + \frac{X(\mathbf{x};\,\boldsymbol{\delta})}{m} + g\sin\theta,$$

$$\dot{v} = pw - ru + \frac{Y(\mathbf{x};\,\boldsymbol{\delta})}{m} - g\cos\theta\sin\phi,$$

$$\dot{w} = qu - pv + \frac{Z(\mathbf{x};\,\boldsymbol{\delta})}{m} - g\cos\theta\cos\phi,$$

$$\dot{\phi} = p + q\sin\phi\,\tan\theta + r\cos\phi\,\tan\theta,$$

$$\dot{\theta} = q\cos\phi - r\sin\phi,$$

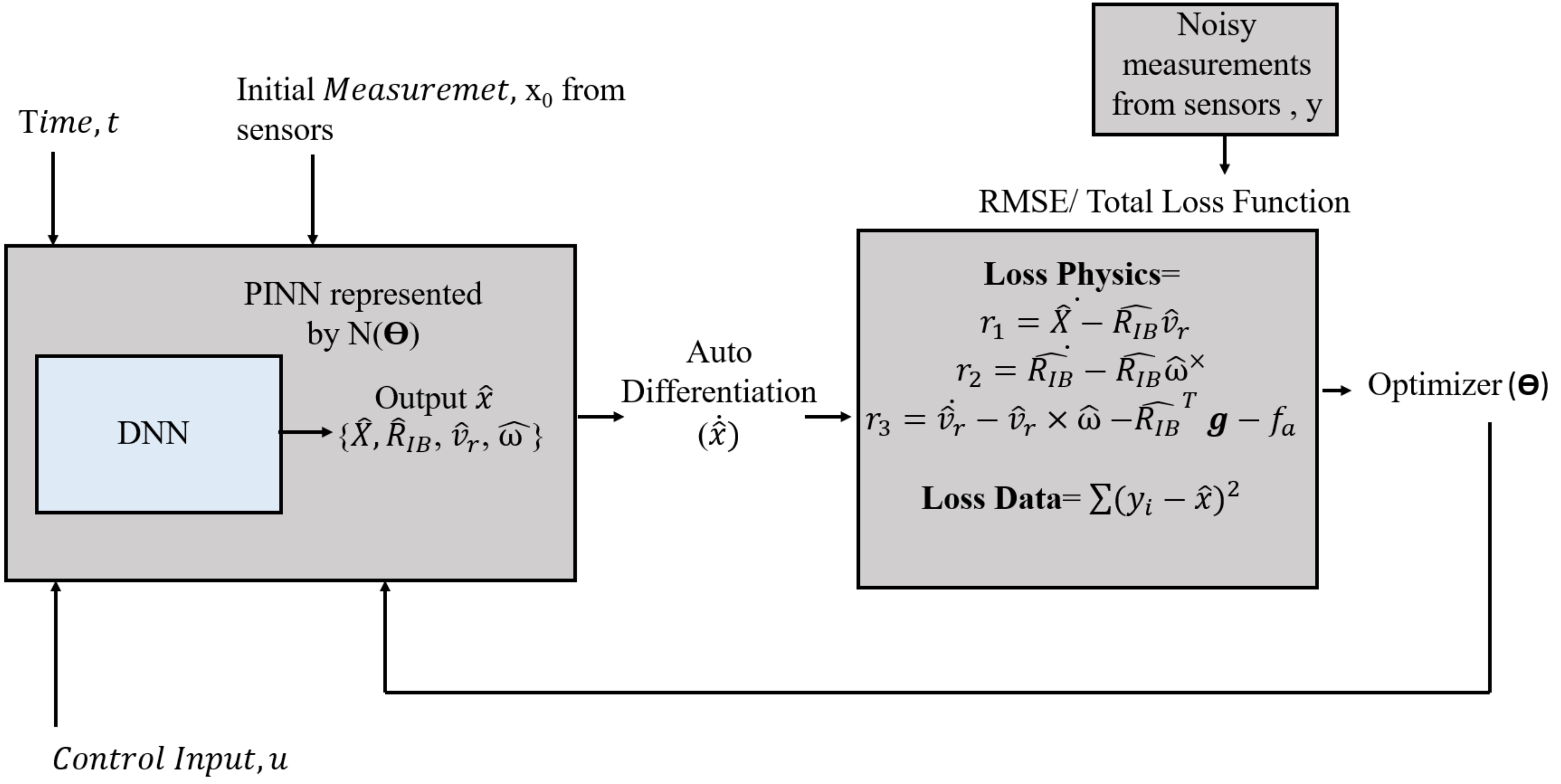
$$\dot{\psi} = \frac{q\sin\phi + r\cos\phi}{\cos\theta},$$

$$\dot{p} = \frac{1}{I_x}\Big(L(\mathbf{x};\,\boldsymbol{\delta}) + (I_y - I_z)\,q\,r\Big),$$

$$\dot{q} = \frac{1}{I_y}\Big(M(\mathbf{x};\,\boldsymbol{\delta}) + (I_z - I_x)\,p\,r\Big),$$

$$\dot{r} = \frac{1}{I_z}\Big(N(\mathbf{x};\,\boldsymbol{\delta}) + (I_x - I_y)\,p\,q\Big),$$

PINN Architecture



PINN estimation is closer to **batch smoothing** than EKF-style point-wise filtering.

PINN Loss Function

Neural Network Approximation
- A function $X_\theta(t)$ parameterized by neural network weight and biases θ. This function approximates the UAV's state x(t).

Residual of Dynamics
- Physics residual was computed by plugging $X_\theta(t)$ into the UAV's differential equations. Any deviation from zero becomes a penalty in the loss function.

Data Loss:
- Noisy measurements of x(t) were taken from simulation and penalized the deviations from $X_\theta(t)$.

Loss Function
- Loss function consists of both data loss and physics loss.

For the purpose of this study, I have chosen MSE, as my error function. It has two components. First one refers the data mismatch and second one for physics mismatch.

$$MSE = MSE_{\hat{y}} + \gamma MSE_{l}.$$

$$MSE_{\hat{y}} = \frac{1}{N_{\hat{y}}} \sum_{i=1}^{N_{\hat{y}}} \frac{1}{N_t} \sum_{j=1}^{N_t} |\hat{y}_i(t^j) - \hat{y}^j_{i,ref}|^2$$

$$MSE_{l} = \frac{1}{N_{\hat{y}}} \sum_{i=1}^{N_{\hat{y}}} \frac{1}{N_l} \sum_{j=1}^{N_l} |l(\hat{y}_i(t^k))|^2$$

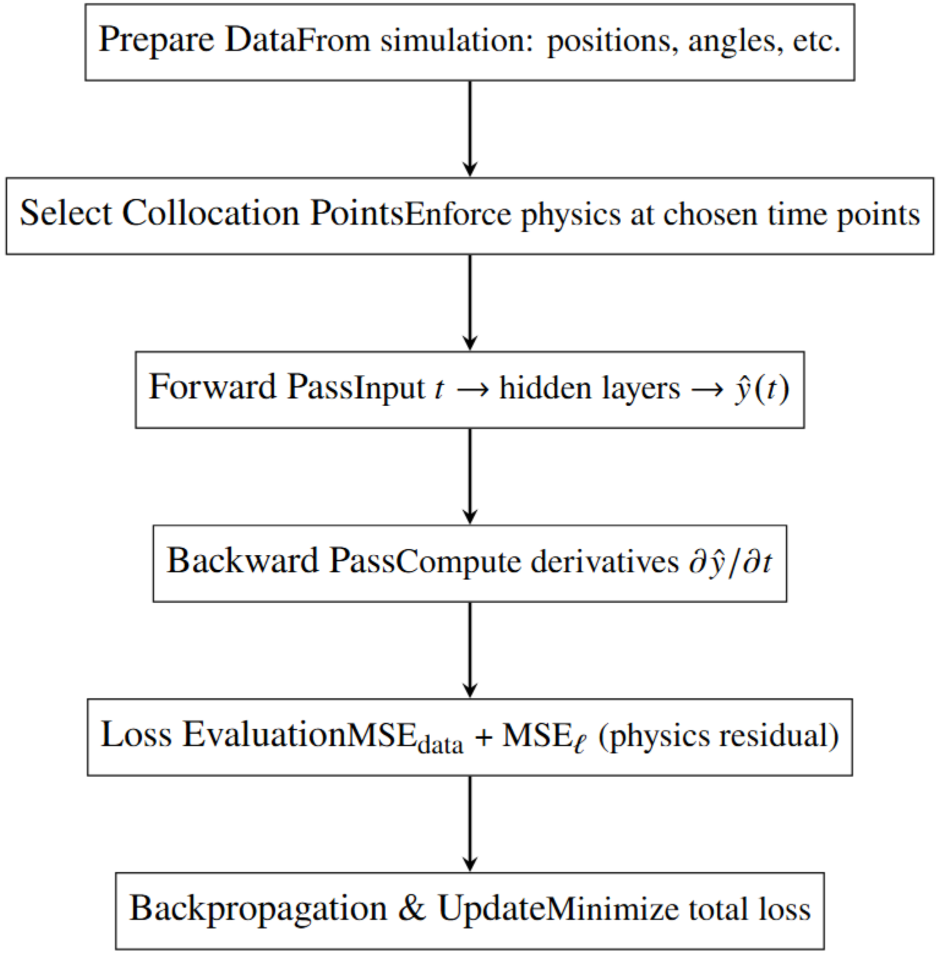
There are other ways to define the error. Here are some examples.

$$MAE = \frac{\sum_{i=1}^{N} |y_{ref,i} - y_i|}{N}, \quad MSE = \frac{1}{N} \sum_{i=1}^{N} (y_{ref,i} - y_i)^2$$

$$ISE = \int (y_{ref} - y)^2 dt, \quad IAE = \int |\, y_{ref} - y \,| \, dt$$

$$ITAE = \int (t \,|\, y_{ref} - y \,|) dt$$

PINN Implementation Steps



Flowchart: PINN-Based Fixed-wing State Estimation

Hyperparameters

Hyperparameters	Option explored	Option selected
Learning Rate	0.015	0.015
Epochs	1000-5000 in thousand increment	5000
Batch Size	256,512,1024	256
Hidden Layers	4,6,8	4
Optimizer	Adam	Adam and L-BFGS-B
Activation Function	Relu, sigmoid, tanh	tanh
Batch Normalization	ON/OFF	ON
Number of Neurons per layer	50,100,200	100

Key points to consider

- Data Normalization
- Usage of optimizer according to the need
- Physics informed learning needs careful lambda hyperparameter choice
- Bayesian Optimization can be considered for hyperparameter tuning

Two Stage Optimization Strategy

Why two stages?
•PINN loss is **non-convex and stiff** (data + physics terms)
•A single optimizer is insufficient for both exploration and fine convergence

Stage 1: Adam (Coarse Training)
•Used for **initial training / early convergence**
•First-order optimizer (gradient-only)
•Adaptive learning rate per parameter (momentum + RMSProp)
•**Robust to noisy gradients and sparse / irregular collocation points**
•Quickly drives the solution toward a good region of parameter space

Stage 2: L-BFGS-B (Fine Tuning)
•Applied after Adam converges
•Uses second-order curvature information (approximate inverse Hessian)
•**Refines the solution to high accuracy**
•Enforces parameter bounds when needed

Initialize network parameters
↓
Stage 1: Adam Optimizer
• Fast exploration of parameter space
• Handles noisy gradients
• Stabilizes competing data & physics losses
↓
Switch optimizer (warm start)
↓
Stage 2: L-BFGS-B Optimizer
• Uses curvature information
• Sharpens convergence
• Enforces physics precision
↓
Final PINN state estimate

Adam: fast exploration → L-BFGS-B: precise refinement

Optimizer Choice

$$\text{Adam: } \boldsymbol{\theta}_{t+1} \sim \frac{\text{momentum}}{\text{variance}} \quad \Big| \quad \text{L-BFGS-B: } \boldsymbol{\theta}_{k+1} \sim \mathbf{H}^{-1}\nabla\mathcal{L}$$

Aspect	Adam	L-BFGS-B
Optimization type	First-order	Quasi-Newton
Information used	Gradient only	Gradient + curvature
Primary role	Global exploration	Local refinement
Behavior in PINNs	Stabilizes early training	Enforces physics precision
Sensitivity to noise	Low	Higher
Convergence speed	Fast (early)	Fast (near optimum)
Constraint handling	Not native	Box constraints supported
When used	Beginning of training	End of training

Equations that define Adam

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t \quad \text{(1st moment: momentum)}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2 \quad \text{(2nd moment: variance)}$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \varepsilon}$$

Equations that define L-BFGS-B

$$\mathbf{p}_k = -\mathbf{H}_k^{-1}\mathbf{g}_k \quad \text{(Newton-like direction)}$$

$$\mathbf{s}_k = \boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k, \quad \mathbf{y}_k = \mathbf{g}_{k+1} - \mathbf{g}_k$$

$$\mathbf{H}_k^{-1} \approx \text{LBFGS}(\{\mathbf{s}_i, \mathbf{y}_i\}_{i=k-m}^{k})$$

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \alpha_k \mathbf{p}_k, \quad \mathbf{l} \leq \boldsymbol{\theta} \leq \mathbf{u}$$

State-space coverage progressively

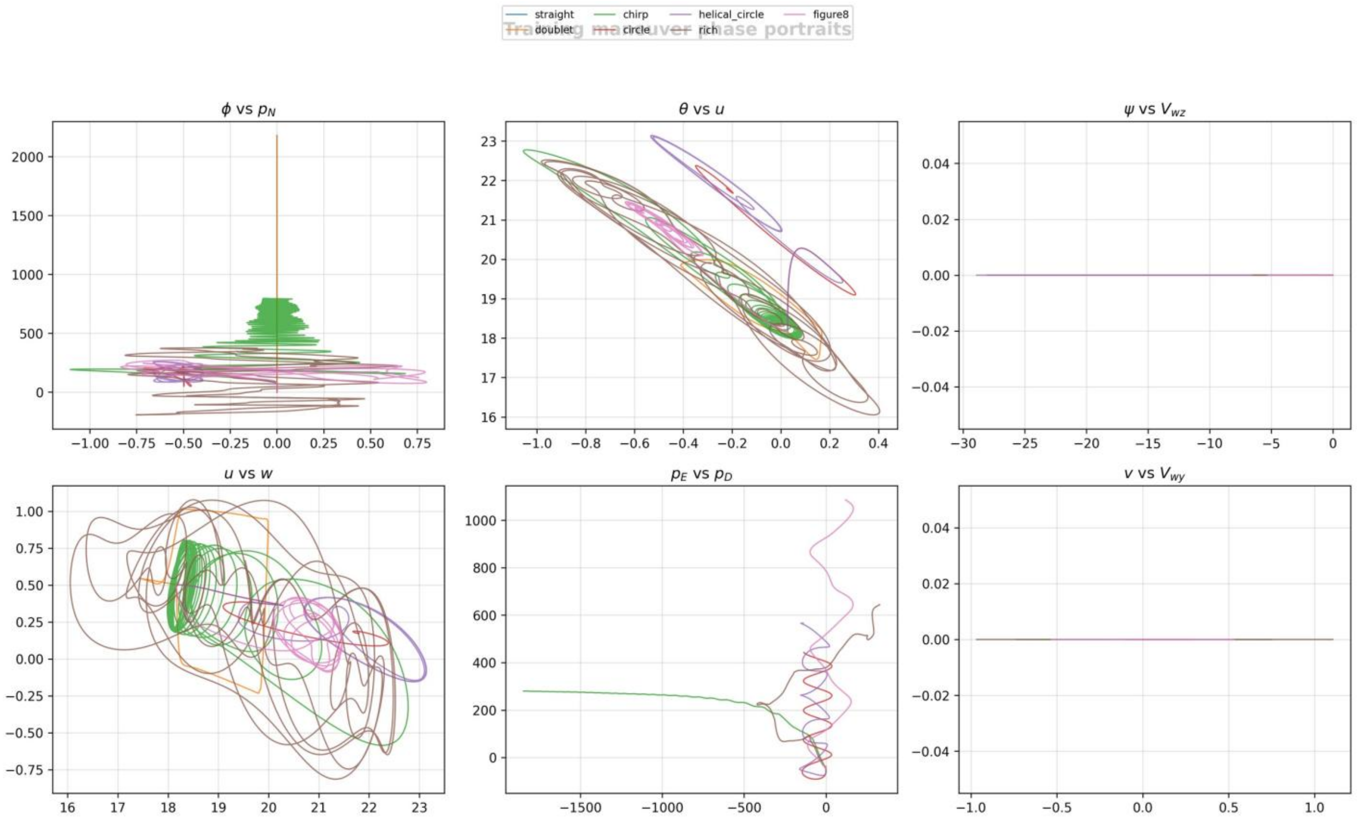


Fig. 5. Training phase portraits across selected state pairs.

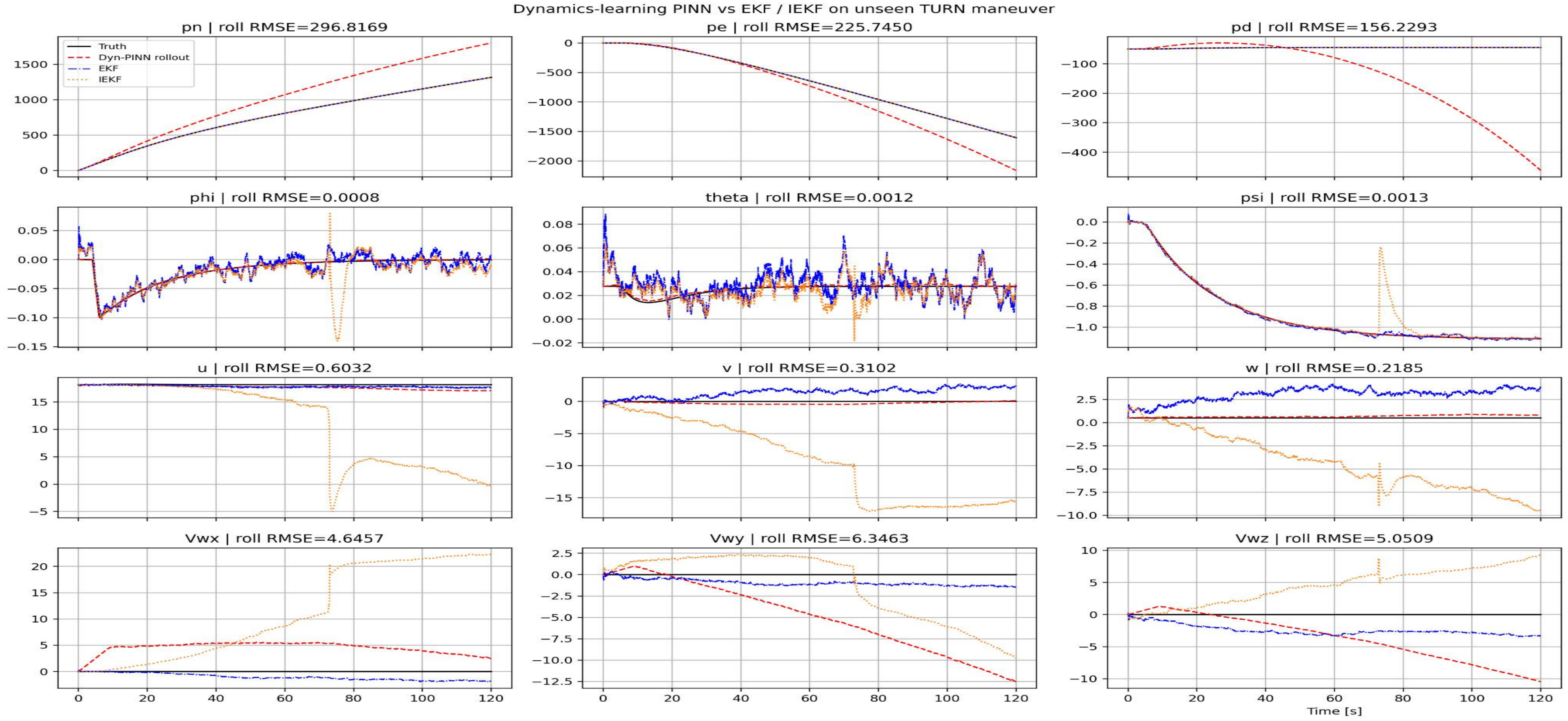
SEVEN CURRICULUM PHASES

- 1: straight
- 2: + doublet
- 3: + chirp
- 4: + circle
- 5: + helical circle
- 6: + rich multi-sine
- 7: + figure-eight

Phase k loss schedule:
λ_e = min(1, (e+1)/75)

Each phase:
350 Adam epochs → restore best val → L-BFGS-B fine tuning

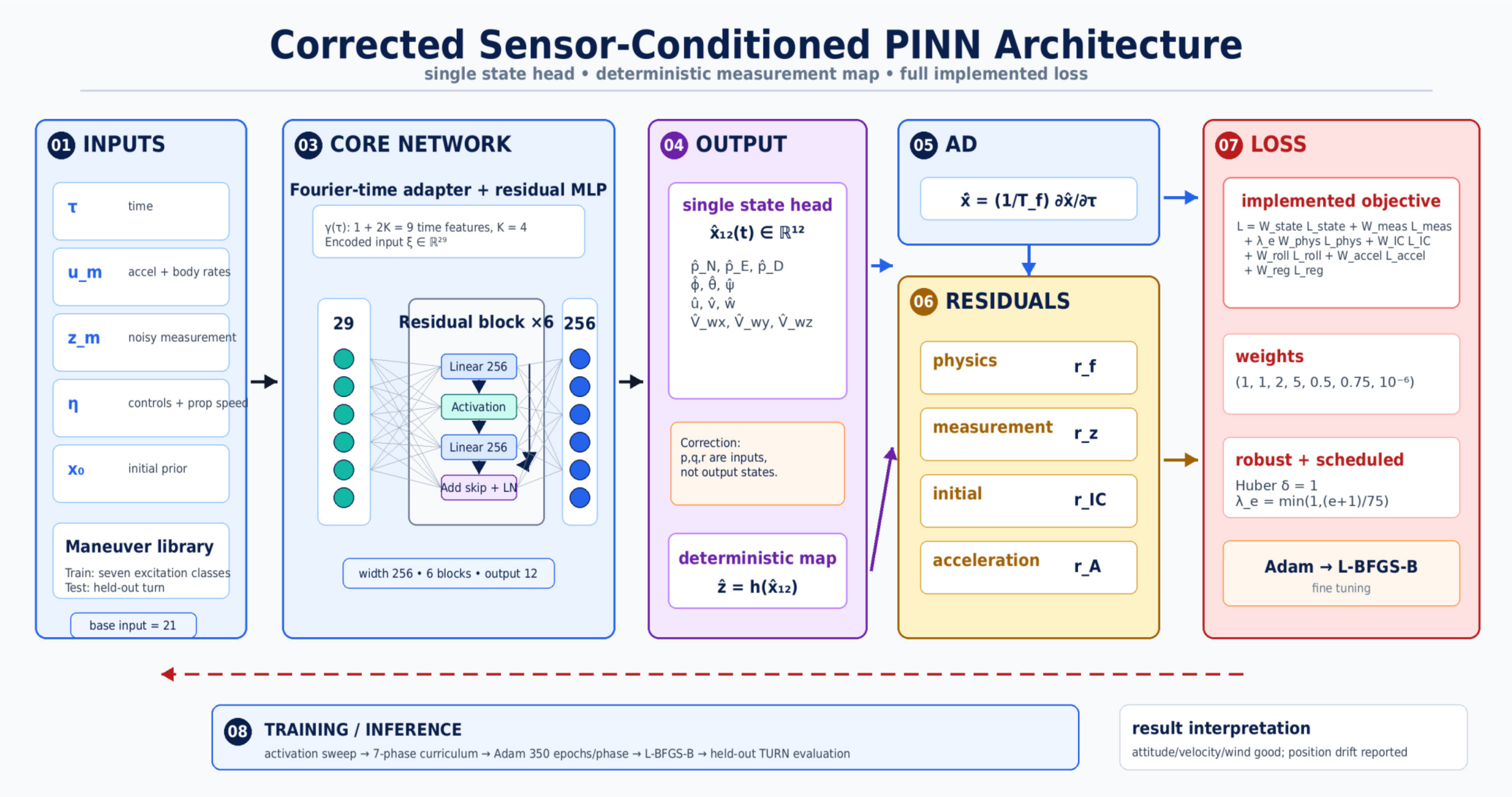
Unseen TURN result: dynamics-learning rollout vs EKF/IEKF



Dynamics-learning rollout is red dashed; EKF is blue dash-dot; IEKF is orange dotted. Subplot titles report rollout RMSE.

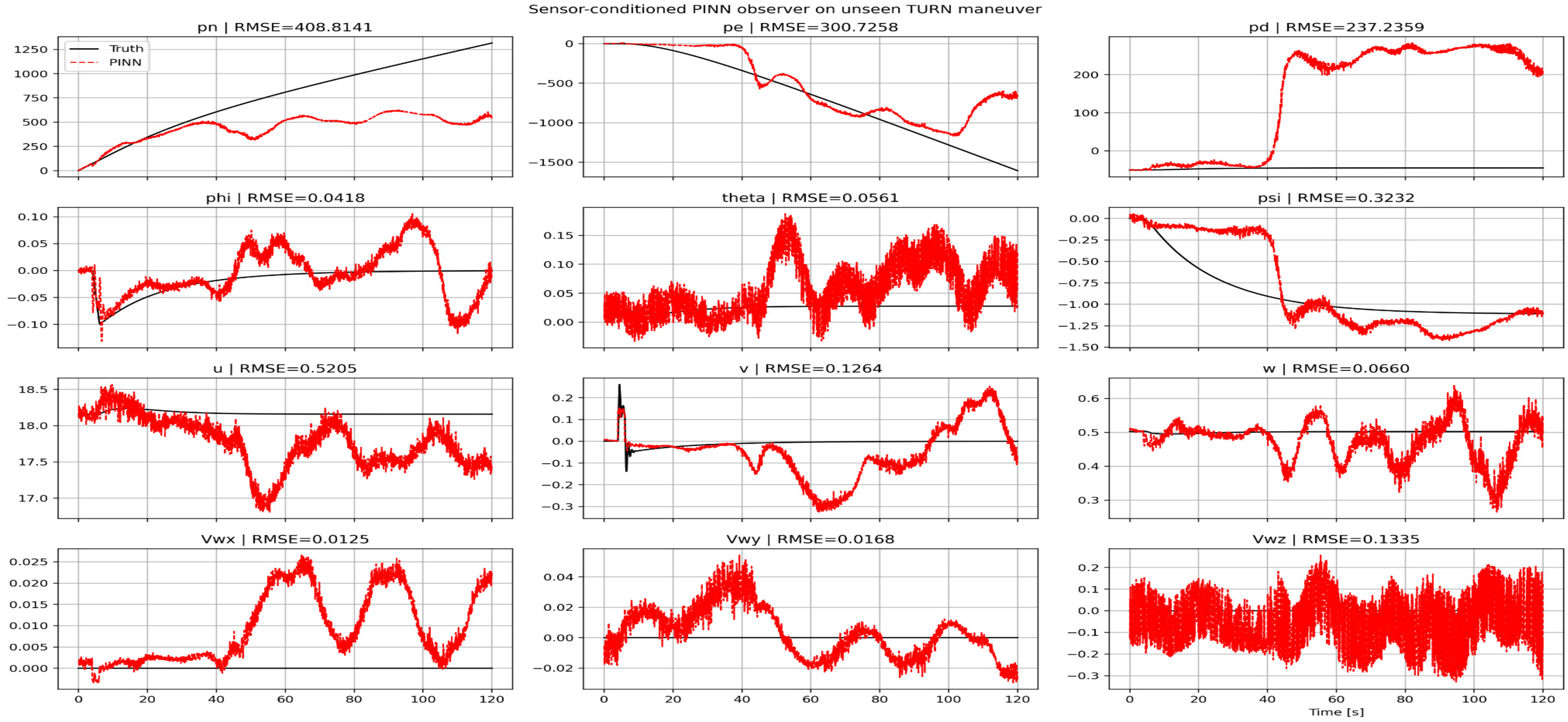
Sensor Conditioned PINN architecture figure

This slide replaces the earlier inconsistent architecture image while preserving the same high-level visual structure.



Key correction: the model has one trainable state head; the measurement branch is analytic, and the output state includes V_wx,V_wy,V_wz instead of p,q,r.

Unseen TURN result: sensor-conditioned PINN states



Direct PINN estimates are plotted in red against simulator truth in black. Position channels dominate the visible error; wind channels remain small.

Empirical local observability diagnostic supports state identifiability

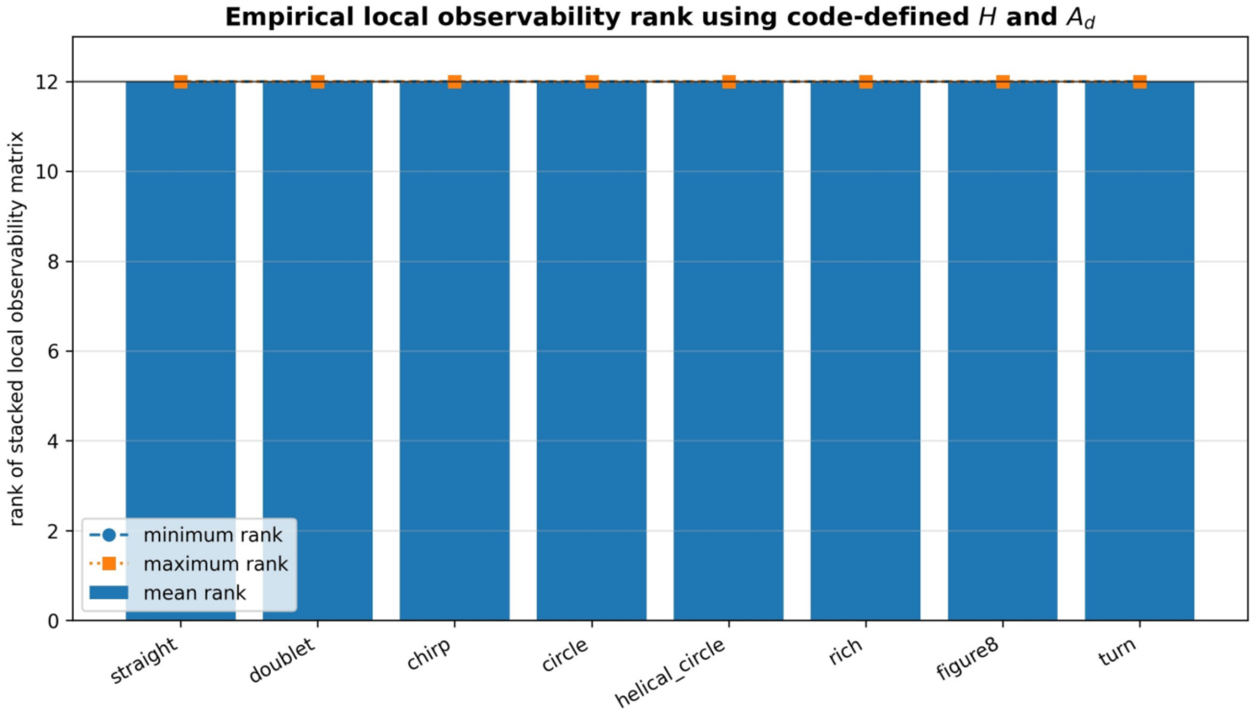


Fig. 6. Finite-difference observability rank for all training maneuvers and the held-out turn.

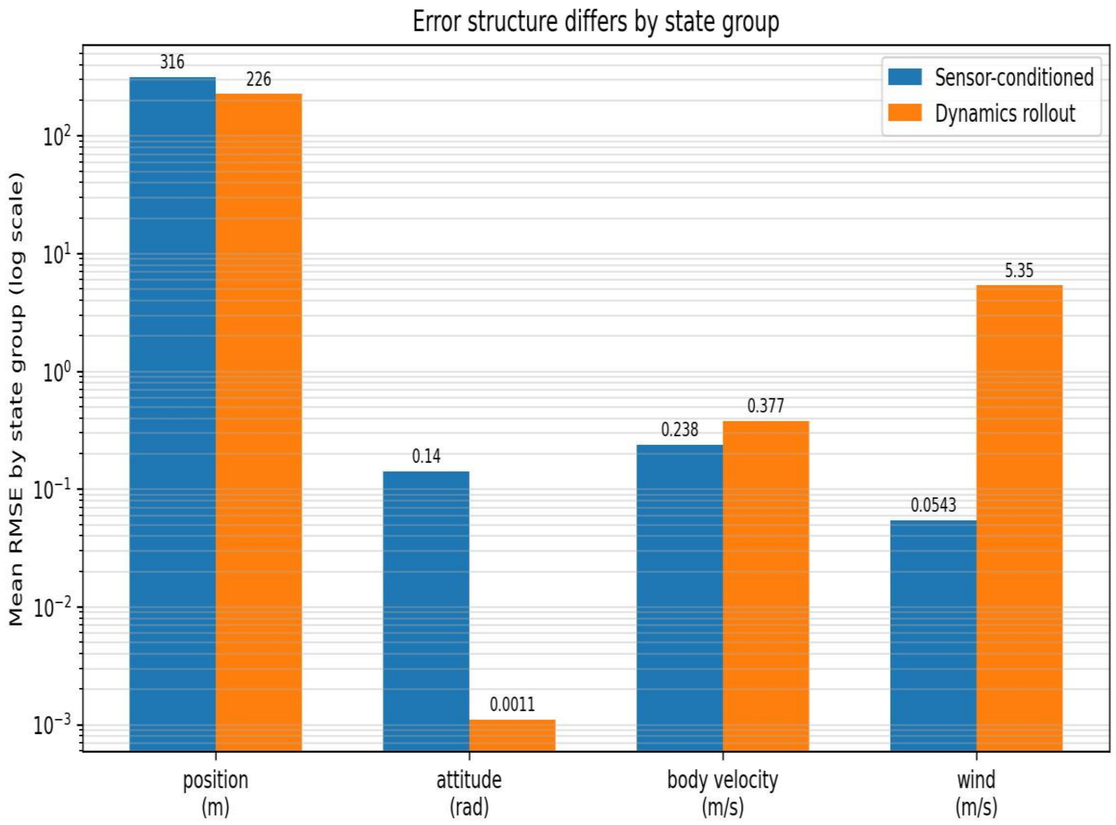
Finite-difference linearization:
$A = \partial f\_o/\partial x, \quad H = \partial h/\partial x$
$A\_d = I + T\_s A$

Stacked local observability matrix:
$O = [\, H; H\ A\_d; H\ A\_d^2; \ldots; H\ A\_d^{n-1} \,]$

Diagnostic criterion:
$\mathrm{rank}(O) = 12$ for the reduced 12-state observer model

Interpretation: the diagnostic is local and numerical; it does not prove global observability, but it indicates that the implemented measurement map is informative enough along the sampled trajectories.

Direct PINN-to-PINN comparison by state



Conclusion

Effect of physics weighting (λ)

Moderate λ balances data fitting and physics consistency

Higher λ improves trajectory consistency when data are heavily noise-corrupted

Main conclusions

- PINNs successfully reconstruct multiple UAV states **without explicit noise modeling**
- Embedding equations of motion acts as a **continuous regularizer**
- Improved fidelity for **unobserved or weakly observed states**
- Robust performance under noisy sensor conditions

Next steps

Conclusion

The two PINN codes solve complementary tasks. The dynamics-learning PINN is stronger for temporal state propagation; the sensor-conditioned PINN is stronger for measurement-conditioned wind and velocity reconstruction.

1. Best current use

Use the dynamics-learning model for rollout-aware process correction, especially where temporal consistency dominates.

2. Best current use

Use sensor conditioning for algebraic measurement consistency, wind recovery, and direct state reconstruction from noisy sensor channels.

3. Next architecture

Build a hybrid: sensor-conditioned transition correction inside a recursive filter with uncertainty and rollout training.

Recommended next experiment: train a recurrent sensor-conditioned transition PINN with measurement updates at every step, compare against EKF/IEKF using the exact saved RMSE arrays, and add ablations for Fourier features, rollout horizon, and physics weights.

Limitations and Future Study

Observed limitations

Reduced accuracy during **highly nonlinear transient maneuvers**

Sensitivity to **collocation point distribution**

Computational cost limits the high fidelity data collection

Assumes the knowledge of system's governing equation

Future direction

Incorporate model uncertainty and real flight data

Online / streaming PINN formulations

Extend from **9-state to full 12-state estimation**

Toward hybrid comparisons with EKF and Invariant EKF

Limitations	Further Improvement
Simulation based synthetic data	Real life data
Static Data	Online stream of data
Collocation point	High number of data point
9 states estimation	12 states estimation

Thank You for Your Patience
Questions and Answers